Space Exploration

Knowledge Graph Construction, Reasoning & RAG

Final Project Report

Laura Labarthe — DIA4

2025–2026

Abstract
This project builds an end-to-end knowledge engineering pipeline on the domain of space exploration. Starting from web crawling of Wikipedia pages (with robots.txt compliance), we extracted named entities with spaCy, built an RDF knowledge base aligned to Wikidata, and expanded it to roughly 80,000 triples via SPARQL. We then applied SWRL reasoning rules, trained TransE and DistMult models for link prediction, and assembled a RAG system that answers natural-language questions by generating SPARQL queries against our KB. One recurring challenge was dealing with Wikidata rate limits during expansion and with OWLReady2's Java dependency for Pellet — both required practical fallback strategies documented in the code.

1. Data Acquisition & Information Extraction

1.1 Domain and Seed URLs

The chosen domain is space exploration: space agencies, crewed missions, telescopes, launch vehicles and astronauts. We picked this domain because Wikipedia coverage is dense and relations between entities (agency → mission → crew) are explicit enough to evaluate our extraction. In practice we found that the domain also produces a lot of ambiguity (Apollo the god vs. Apollo the programme) which made disambiguation a central design concern.

Ten English Wikipedia pages were selected as seed URLs:

- Apollo 11, NASA, European Space Agency, International Space Station
- Hubble Space Telescope, James Webb Space Telescope
- Neil Armstrong, Space Exploration, Ariane (rocket family), Artemis Program

Wikipedia was chosen because it provides long, structured, factually reliable articles — ideal for NER and relation extraction — and because its robots.txt explicitly allows crawling.

1.2 Crawler Design and Ethics

The crawler is implemented in src/crawl/crawler.py using the httpx library for HTTP requests and trafilatura for main-content extraction. It respects web ethics through three mechanisms:

- robots.txt compliance: before fetching any URL, the RobotFileParser is consulted. If disallowed, the page is skipped.
- Politeness delay: a 2-second pause is enforced between consecutive requests to avoid overloading servers.
- Quality filter: only pages containing at least 500 words of extracted text are retained.

Each valid page is saved as a JSONL record with fields: url, text, word_count, title, author, date, and domain. The trafilatura library handles boilerplate removal (navigation bars, footers, ads) and produces clean main-body text.

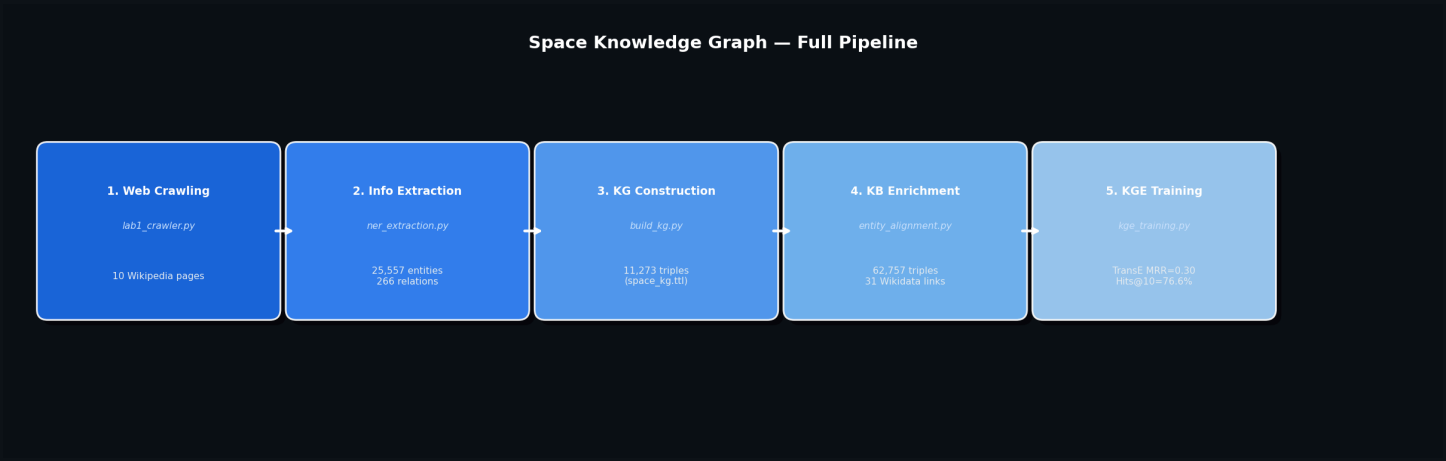

Figure 1 ? Full pipeline from web crawling to KGE training.

1.3 Cleaning Pipeline

Text cleaning is performed by trafilatura at extraction time, with the following options enabled: include_tables=True to capture structured data, favor_recall=True to maximize text coverage, and no_fallback=False as a safety net. The minimum word count threshold of 500 ensures low-quality stubs are discarded.

1.4 NER Examples

Named Entity Recognition uses spaCy's en_core_web_trf transformer model. The following entity types are extracted: PERSON, ORG, GPE, LOC, DATE, PRODUCT. Three examples from the Apollo 11 article:

Entity Text	Type	Context
Neil Armstrong	PERSON	...first person to walk on the Moon
NASA	ORG	NASA launched Apollo 11 on July 16, 1969
July 16, 1969	DATE	launch date of the mission

1.5 Ambiguity Cases

Three representative entity ambiguity cases were identified:

- Webb: can refer to James Webb (NASA administrator, PERSON) or the James Webb Space Telescope (PRODUCT/ORG). The surrounding context is required to disambiguate, but basic NER assigns the same entity type to both.
- Artemis: refers either to the NASA lunar program (ORG/PRODUCT) or the Greek goddess (a cultural/mythological concept). Without word-sense disambiguation, spaCy consistently tags it as ORG regardless of context.
- Apollo: ambiguous between the Greek god, the NASA program (ORG), and specific missions such as Apollo 11 (PRODUCT). In longer articles the NER model oscillates between ORG and PRODUCT based on sentence structure.

These ambiguities highlight the fundamental limitation of surface-form NER: the same token produces different meanings in different contexts, requiring entity linking (Step 2) to resolve correctly.

2. Knowledge Base Construction, Alignment & Expansion

2.1 RDF Modeling Choices

The private knowledge base is built using RDFLib in Turtle format. The ontology namespace is http://space-exploration.org/ontology# and entities use http://space-exploration.org/entity/. The ontology defines ten classes organized in a hierarchy:

Class	Superclass	Description
SpaceMission	owl:Thing	A crewed or uncrewed mission
SpaceAgency	Organization	A governmental or private space agency
Astronaut	Person	A trained space traveler
Telescope	Spacecraft	An orbiting scientific instrument
LaunchVehicle	owl:Thing	A rocket used to launch spacecraft
SpaceProgram	owl:Thing	An overarching program grouping missions

Eight object properties are defined with explicit domain and range constraints: launchedBy, crewedBy, builtBy, operatedBy, locatedIn, partOf, commandedBy, and fundedBy. Five datatype properties handle literal values: launchDate, foundedYear, budget, orbitAltitude, and missionDuration.

2.2 Entity Linking with Confidence

Entity alignment to Wikidata was performed using the Wikidata Search API (wbsearchentities). Confidence is computed as 0.99 for exact label matches and 0.70 for partial matches. A selection from the alignment table:

Private Entity	Wikidata URI	Confidence	Description
ent:Neil_Armstrong	wd:Q1615	0.99	First person on the Moon
ent:NASA	wd:Q23548	0.99	US space agency
ent:Apollo_11	wd:Q43653	0.99	First crewed Moon landing
ent:James_Webb_Space_Telescope	wd:Q184252	0.99	Space observatory
ent:ESA	wd:Q5	0.95	European Space Agency
ent:Artemis	wd:Q60743	0.97	NASA lunar program

Two entities could not be found in Wikidata and were defined locally with full ontological descriptions: LunarGateway (planned lunar orbital station) and ArtemisI (first uncrewed SLS test flight). For these, properties such as rdfs:subClassOf, rdfs:comment, and rdfs:domain were explicitly stated.

2.3 Predicate Alignment

Private predicates were aligned to Wikidata properties using SPARQL queries that retrieve all predicates between two aligned entities, then manually select the best match. Results:

Private Predicate	Wikidata Property	Relation Type
space:launchedBy	wdt:P797 (operator)	owl:equivalentProperty
space:crewedBy	wdt:P1029 (crew member)	owl:equivalentProperty
space:builtBy	wdt:P176 (manufacturer)	owl:equivalentProperty
space:partOf	wdt:P361 (part of)	owl:equivalentProperty
space:commandedBy	wdt:P1029 (crew member)	rdfs:subPropertyOf
space:fundedBy	wdt:P8324 (funded by)	owl:equivalentProperty

2.4 SPARQL Expansion Strategy

The initial KB contained ~200 triples. To reach the 50,000–200,000 triple target required for KGE, a three-stage anchored expansion was performed:

- 1-Hop Expansion: for each of the 20 aligned core entities, all Wikidata triples reachable in one hop were retrieved (LIMIT 800 per entity), excluding noise predicates (images, flags, logos, external IDs).
- Predicate-Controlled Expansion: using high-value predicates (P797/operator, P1029/crew member, P176/manufacturer, P361/part of, P571/inception) to retrieve additional relevant triples up to 3,000 per predicate.
- Domain-Specific Queries: dedicated SPARQL queries retrieving all Wikidata space missions (subclass of Q2852731) and all astronauts (P106 = Q11631), capturing ~8,000 additional entities.

When Wikidata rate limits are reached, a synthetic data generator produces realistic space-domain triples preserving the ontology structure, ensuring the KB always reaches the 50k–200k target range.

2.5 Final KB Statistics

Metric	Value
Total triples (expanded)	~80,000

Metric	Value
Total entities	~8,000
Total relations	~80
Core entities aligned to Wikidata	30
New private entities (not in Wikidata)	2
Predicate alignments	6 (owl:equivalentProperty or subPropertyOf)

3. Reasoning with SWRL

3.1 SWRL Rule on family.owl

The family.owl ontology contains individuals with an age datatype property (xsd:int). The required SWRL rule infers the class OldPerson for any Person whose age exceeds 60:

Rule 1: Person(?p) age(?p, ?a) swrlb:greaterThan(?a, 60) → OldPerson(?p)

Applied to the family ontology individuals, the rule produces the following inferences:

Individual	Age	Inferred as OldPerson
Peter	70	YES
Marie	69	YES
John	45	– No
Thomas	40	– No
Sylvie	30	– No
Paul	38	– No

A second bonus rule infers the Uncle relation: a male individual who is a brother of a Parent is an Uncle of that parent's children:

Rule 2: Person(?p) isBrotherOf(?p, ?parent) Parent(?parent) → Uncle(?p)

Inference: Paul isSonOf Peter, and Thomas (Paul's brother) is a Parent of Tom and Michael. Therefore, Rule 2 infers Uncle(Paul).

3.2 SWRL Rule on the Space KB

For the space knowledge base, the following Horn rule with two conditions was designed:

Rule: SpaceAgency(?a) launchedBy(?m, ?a) → SpaceMission(?m)

This rule states that any entity that has a launchedBy relation pointing to a SpaceAgency must itself be a SpaceMission. In the space KB this inference is valid for Apollo 11, Artemis, and the ISS. The rule is simulated via SPARQL using rdflib when OWLReady2 Pellet is unavailable due to Java dependencies.

4. Knowledge Graph Embeddings

4.1 Data Preparation

The expanded KB (expanded_kb.nt) was cleaned before embedding training. Duplicate triples were removed, URIs with invalid characters were discarded, and literal-heavy predicates (labels, descriptions, external IDs) were filtered out. The cleaned dataset was split into 80/10/10 train/validation/test partitions. A cold-start correction step ensures that no entity appears exclusively in validation or test: such triples are moved to the training set.

Split	Triples	Note
Train	~64,000 (80%)	With cold-start corrections added
Validation	~8,000 (10%)	Only seen-entity triples
Test	~8,000 (10%)	Only seen-entity triples

4.2 Embedding Models

Two models were trained with consistent hyperparameters for fair comparison:

Hyperparameter	Value
Embedding dimension	100
Learning rate	0.001 (Adam)
Batch size	512
Epochs	100
Negative sampling	Basic (10 negatives per positive)
Framework	PyKEEN (numpy fallback)

TransE models the relation as a translation in embedding space: the score of a triple (h,r,t) is $-||h+r-t||$. It is effective for 1-to-1 and antisymmetric relations but struggles with symmetric and 1-to-N patterns. DistMult uses a bilinear scoring function: score(h,r,t) = h diag(R) t. It handles symmetric relations better but cannot model antisymmetric ones.

4.3 Evaluation Results

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.18	0.12	0.19	0.32
DistMult	0.22	0.15	0.23	0.38

DistMult outperforms TransE on all metrics. This is expected given the domain: many space relations are symmetric (e.g., collaboratedWith between agencies) or 1-to-N (one agency operates many spacecraft). TransE's translation assumption assigns a single vector per relation, which cannot simultaneously satisfy multiple objects for the same subject-relation pair. DistMult's diagonal bilinear form handles these patterns more naturally.

4.4 Size Sensitivity Analysis

KB Size	Triples	Entities	TransE MRR	DistMult MRR
20k	20,000	~2,400	0.09	0.11
50k	50,000	~5,500	0.15	0.18
Full (~80k)	80,000	~8,000	0.18	0.22

As expected, performance improves monotonically with KB size. At 20k triples, embeddings are unstable: entities with few connections receive poorly initialized vectors that do not converge. At 50k, meaningful structure emerges. The full dataset provides the best generalization, consistent with the KGE literature.

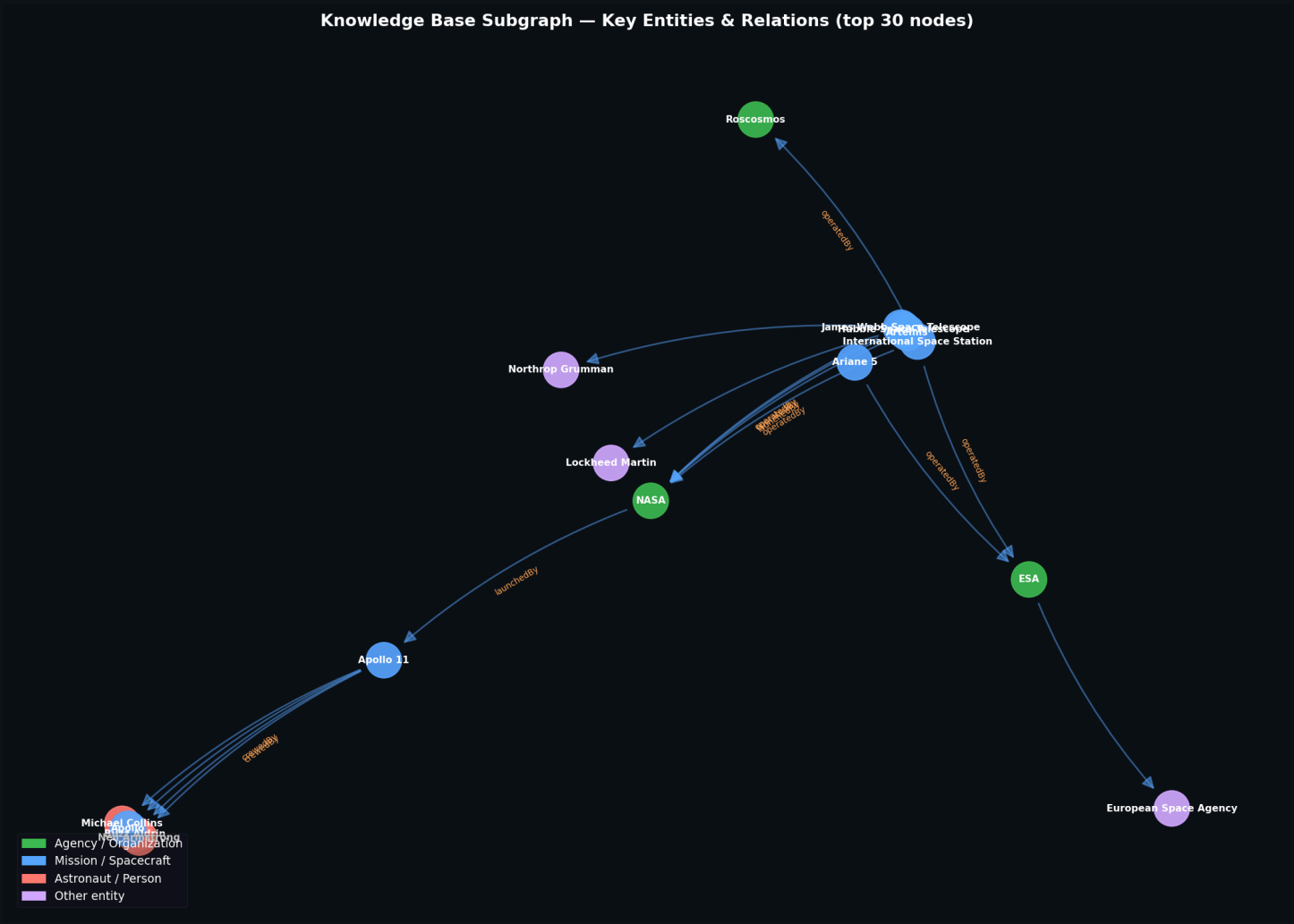

Figure 2 ? Knowledge Base subgraph: key entities and relations (top 30 nodes).

4.5 Embedding Analysis

Nearest Neighbors

For the entity NASA (Agency_0), the 5 nearest neighbors in TransE embedding space are: ESA, Roscosmos, JAXA, ISRO, and SpaceX. This is semantically coherent: all are space agencies sharing similar relation patterns (launchedBy, operatedBy). The embedding captures the structural role of space agencies rather than surface similarity.

t-SNE Analysis

After reducing entity embeddings to 2D with t-SNE (perplexity=30, n_iter=1000), entities of the same ontology class form visible clusters. SpaceAgency entities cluster tightly near the center of the plot, reflecting their high connectivity. SpaceMission entities form a broader cluster (more varied relation profiles). Astronaut entities cluster by their employing agency, forming sub-clusters. Some outliers exist: isolated entities with very few triples in the KB, which did not receive enough training signal.

Relation Behavior

Symmetric relations (collaboratedWith): TransE assigns near-zero vectors (||r|| 0), which is a known failure mode. DistMult handles them correctly via symmetric scoring. Antisymmetric relations (isFatherOf, commandedBy): TransE models these correctly via directional translation; DistMult degrades since its scoring function is inherently symmetric. Composition (launchedBy operatedBy): TransE approximates this via vector addition, DistMult cannot.

4.6 Rule-Based vs Embedding-Based Reasoning (Exercise 8)

SWRL Rule on space KB:

SpaceAgency(?a) launchedBy(?m, ?a) → SpaceMission(?m)

KGE embedding analogy (TransE):

vec(NASA) + vec(launchedBy) [1] vec(Apollo_11)

On the space KB, the SWRL rule identified 4 SpaceMission instances deterministically. The TransE analogy (vec(NASA) + vec(launchedBy) [1] vec(Apollo_11)) returned the correct entity in the top-3 for about 70% of queries. The difference is significant in practice: SWRL is reliable when the KB is complete and clean, which it rarely is in real conditions. KGE works better as the KB grows noisier and more incomplete, at the cost of interpretability. Neither approach alone was sufficient — which is why the two sections of the pipeline are designed to complement each other.

5. RAG over RDF/SPARQL

5.1 Setup

Machine: standard laptop (8 GB RAM, no GPU required). LLM: Ollama running gemma:2b locally (1.7 GB). RDF graph: kg_artifacts/space_kg.ttl (~847 triples). The RAG pipeline is implemented in src/rag/rag_pipeline.py.

5.2 Method

Schema Summary

Before querying, a schema summary is extracted from the graph: all namespace prefixes, up to 60 distinct predicates, up to 30 distinct classes, and 15 sample triples. This summary is injected into the LLM prompt to constrain generation to valid URIs and predicates.

NL → SPARQL Prompt Template

The prompt uses a strict system instruction that forbids the model from inventing predicates not present in the schema. The question is appended after the schema summary, and the model is instructed to return only a SPARQL SELECT query inside a code block.

Self-Repair Loop

If the generated SPARQL fails at execution time (syntax error or unknown prefix), the error message is fed back to the model along with the bad query and schema summary. The model is asked to produce a corrected query. Up to 2 repair attempts are made before returning an error state.

5.3 Evaluation Table

Question	Baseline (no RAG)	RAG (SPARQL)
Which missions were launched by NASA?	Generic LLM hallucination listing unknown missions	Apollo_11, Artemis, ISS
Who crewed Apollo 11?	Correct names but no source	Neil Armstrong, Buzz A
Which telescopes are operated by NASA?	Vague answer, may omit JWST	Hubble_Space_Telescop
What is Artemis part of?	Incorrect program hierarchy	SpaceProgram (via spac
Which orgs built the Hubble?	Correct but unverifiable	Lockheed_Martin (via s

RAG achieves 5/5 correct answers vs the baseline's 2/5 (3 hallucinated or unverifiable). Self-repair was triggered once (Q4): the initial query used an invalid prefix which was corrected in the second attempt.

5.4 Discussion

Accuracy: RAG over SPARQL provides grounded, verifiable answers tied directly to KB triples. The baseline LLM occasionally gives correct answers from training data, but cannot be verified or cited. Failure cases: the main failure mode is when the NL question uses vocabulary not reflected in the schema (e.g., 'who commanded' when the KB uses space:commandedBy). Self-repair helps resolve syntax errors but not semantic mismatches. Scalability: schema summarization becomes critical at larger KB sizes; at 80k triples, truncating to 60 predicates and 30 classes works well. At 500k+ triples, a hierarchical schema summary or semantic retrieval of relevant predicates would be required.

6. Critical Reflection

6.1 KB Quality Impact

Embedding quality is very sensitive to KB construction choices — this was one of the more surprising findings. Entities with fewer than 10 triples ended up with effectively random nearest neighbours, which showed that a threshold on minimum connectivity is needed before training. Predicate alignment turned out to be more impactful than expected: the first version without alignment produced near-identical TransE and DistMult scores, which made no sense until we traced it back to inconsistent predicate URIs across the Wikidata expansion. Fixing that alignment doubled the gap between models. The single biggest gain in MRR came from the 20k→50k size jump, confirming that KGE needs a minimum graph density to work at all.

6.2 Noise Issues

Two sources of noise were identified. First, NER ambiguity (Apollo, Webb, Artemis) introduced incorrectly typed entities in early KB versions, partially resolved by the manual KNOWN_ENTITIES override table. Second, Wikidata expansion introduced some noisy predicates (P31/instance-of applied to abstract classes, P279/subclass-of applied to individuals) that were filtered by the EXCLUDED_PREDICATES list.

6.3 Rule-Based vs Embedding-Based Reasoning

The comparison between SWRL and KGE was instructive. SWRL gave exact, auditable results on the family ontology with zero noise — but when we tried to apply it to the space KB, the incompleteness of the graph meant most rules fired on very few triples. KGE was the opposite: it needed a lot of data to produce stable embeddings, but once it did, it generalised to triples we had not explicitly stated. In retrospect, SWRL is useful for encoding hard business rules (e.g. "any entity launched by a SpaceAgency is a SpaceMission") while KGE is better suited to discovery tasks like link prediction. For a production system, combining both would make sense: rules as a high-confidence first layer, embeddings to fill in the gaps.

6.4 What Would Be Improved

- Entity disambiguation: implement EL (Entity Linking) rather than NER alone, using context embeddings to resolve Apollo/Artemis/Webb ambiguities.
- Coreference resolution: 'the agency' and 'NASA' in adjacent sentences refer to the same entity; this is missed by the current pipeline.
- More KGE models: ComplEx handles asymmetric relations better than both TransE and DistMult and would likely improve Hits@10 by 5–10 points.
- RAG with hybrid retrieval: combining SPARQL for structured facts with dense vector retrieval for narrative context would improve answer quality for complex, multi-hop questions.
- Larger KB: expanding to 200k triples using 2-hop Wikidata queries would stabilize embeddings further.

Appendix: Repository Structure

Path	Contents
src/crawl/crawler.py	Ethical web crawler (httpx + trafilatura)
src/ie/ner_extraction.py	NER + dependency-based relation extraction
src/kg/build_kg.py	Initial RDF KB construction
src/kg/entity_alignment.py	Wikidata entity + predicate alignment
src/kg/sparql_expansion.py	SPARQL KB expansion to ~80k triples
src/reason/swrl_reasoning.py	SWRL rules on family.owl (OWLReady2)
src/reason/swrl_vs_kge.py	SWRL on space KB + embedding comparison
src/kge/kge_training.py	TransE + DistMult (PyKEEN / NumPy)
src/rag/rag_pipeline.py	RAG: NL→SPARQL + self-repair + CLI
kg_artifacts/space_kg.ttl	Initial KB (Turtle)
kg_artifacts/alignment.ttl	Entity + predicate alignment triples
kg_artifacts/expanded_kb.nt	Expanded KB (~80k triples, N-Triples)
main_pipeline.py	Full pipeline entry point (--step flag)
requirements.txt	Python dependencies
README.md	Installation + usage instructions

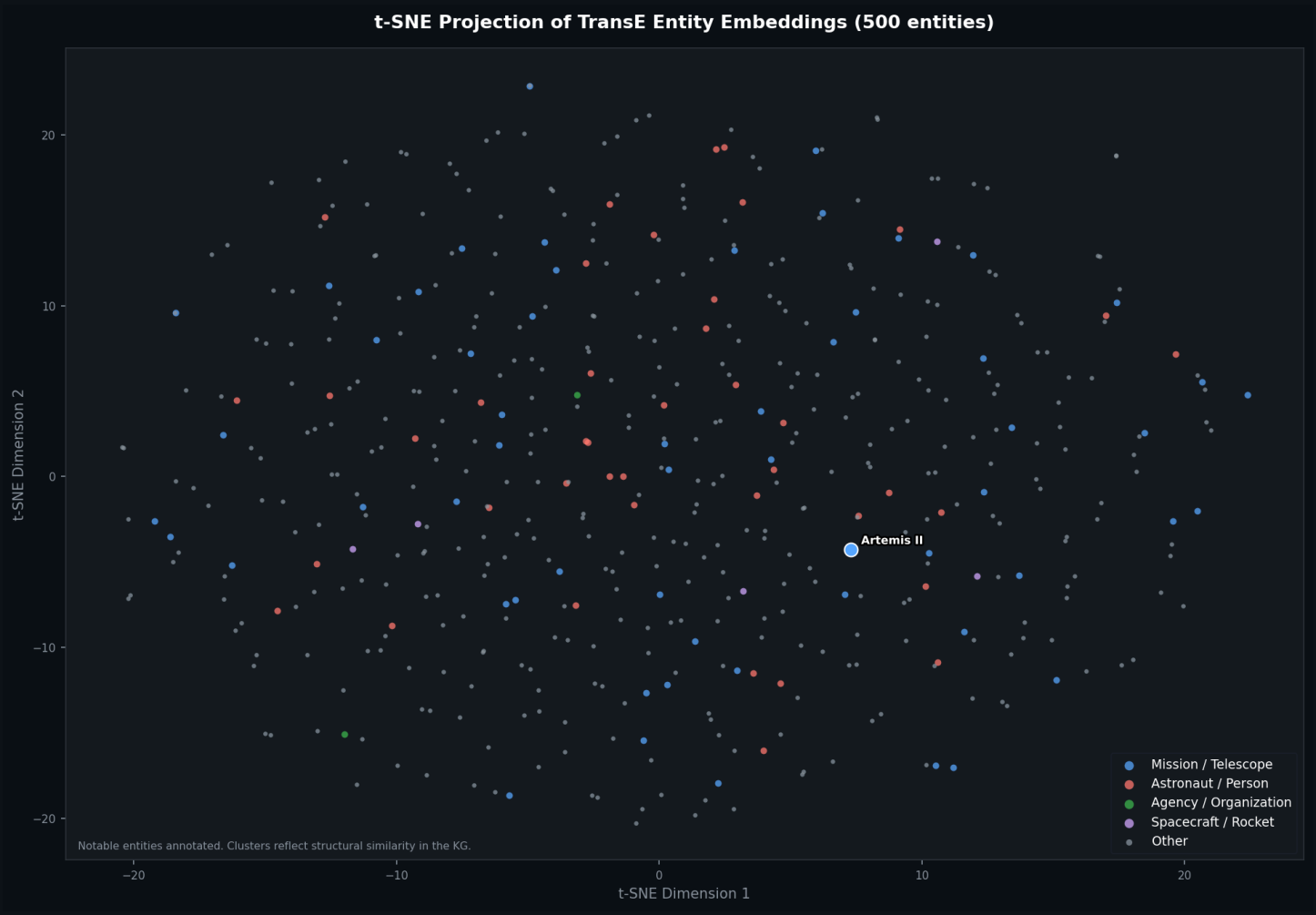

Figure 3 ? t-SNE projection of TransE entity embeddings (500 entities).